

UR5 Robotic Arm

Inverse Kinematics Verification Tool

Project Report
Industrial Training Program on Robotics and AI

Submitted by: Dhruv Khanna

Date: 20 July 2026

1. Problem Statement

This project designs and implements a UR5 Inverse Kinematics Verification Tool that models the UR5 robotic arm using Denavit-Hartenberg (DH) parameters, computes forward kinematics for given joint angles, computes inverse kinematics for a desired end-effector pose, verifies the IK solution using forward kinematics, and computes the manipulator Jacobian to identify singularities. The tool reports a clear PASS/FAIL validity flag for every IK solution.

Deliverables

- Standard DH table and individual homogeneous transforms $A_1 \dots A_6$
- Forward kinematics: base-to-end-effector transform T_{0_6}
- Numerical inverse kinematics solver (damped least-squares)
- Verification of IK solution via FK re-substitution and error metrics
- Geometric Jacobian and singularity analysis

2. DH Parameter Modeling (Question 1)

The UR5 is modelled using the standard DH convention. Each link transform A_i maps frame (i-1) to frame (i):

$$A_i = \text{Rot}_z(\theta_i) \cdot \text{Trans}_z(d_i) \cdot \text{Trans}_x(a_i) \cdot \text{Rot}_x(\alpha_i)$$

Written out in full, the individual homogeneous transform for row i is:

$\cos\theta$	$-\sin\theta \cos\alpha$	$\sin\theta \sin\alpha$	$a \cos\theta$
$\sin\theta$	$\cos\theta \cos\alpha$	$-\cos\theta \sin\alpha$	$a \sin\theta$
0	$\sin\alpha$	$\cos\alpha$	d
0	0	0	1

UR5 DH Parameter Table

Link offsets/lengths are the manufacturer nominal values (metres, radians). $\theta_i = q_i$ is the variable joint angle for each revolute joint.

i	a_i (m)	alpha_i (rad)	d_i (m)	theta_i
1	0.00000	1.57080	0.08916	q_1 (variable)
2	-0.42500	0.00000	0.00000	q_2 (variable)
3	-0.39225	0.00000	0.00000	q_3 (variable)
4	0.00000	1.57080	0.10915	q_4 (variable)
5	0.00000	-1.57080	0.09465	q_5 (variable)
6	0.00000	0.00000	0.08230	q_6 (variable)

Sample evaluation at demonstration joint vector q (deg) = [30.0, -45.0, 60.0, -90.0, 45.0, 20.0], the resulting numerical $A_1 \dots A_6$ matrices (computed by `dh.py` / `dh_transform`) are used directly in Section 3

to build the overall transform. (Full symbolic derivation is provided in the accompanying hand-written calculation sheet.)

3. Forward Kinematics (Question 2)

The overall base-to-end-effector transform is the ordered product of the individual link transforms:

$$T_{06} = A1 \cdot A2 \cdot A3 \cdot A4 \cdot A5 \cdot A6$$

Given joint vector q (deg) = [30.0, -45.0, 60.0, -90.0, 45.0, 20.0]

Resulting T_{06}

0.76727	0.61094	0.19506	-0.59693
-0.32427	0.63198	-0.70388	-0.53787
-0.55330	0.47681	0.68301	0.31987
0.00000	0.00000	0.00000	1.00000

End-effector position (m): $x = -0.59693$, $y = -0.53787$, $z = 0.31987$. The upper-left 3x3 block of T_{06} is the end-effector orientation (rotation matrix); the last column is the position vector, as printed above.

4. Inverse Kinematics (Question 3)

A numerical IK solution is implemented using the Damped Least-Squares (Levenberg-Marquardt) method on the geometric Jacobian. This method was chosen over a closed-form/analytical solution because it is general, numerically robust near singularities (via the damping term), and directly reuses the same Jacobian required by Question on singularity analysis — giving one consistent, verifiable code path.

At each iteration k :

- Compute current pose $T(q_k)$ via forward kinematics.
- Compute the 6x1 pose error $e = [p_{\text{desired}} - p_{\text{current}}; \text{orientation_error}(R_{\text{current}}, R_{\text{desired}})]$, where the orientation error is extracted as an angle*axis vector from $R_{\text{err}} = R_{\text{desired}} R_{\text{current}}^T$.
- Compute the geometric Jacobian $J(q_k)$.
- Solve the damped normal equations: $\Delta q = J^T (J J^T + \lambda^2 I)^{-1} e$.
- Update $q_{k+1} = q_k + \Delta q$ (step clamped for stability) and repeat until $\|e\| < \text{tolerance}$ (1e-6) or the iteration limit is reached.

To improve robustness the solver is run from 8 randomized initial guesses (inverse_kinematics_multi_start) and the best-converged solution is kept.

Demonstration Run

A target pose T_{desired} was generated from a known joint vector q_{true} (deg) = [15.0, -70.0, 100.0, -95.0, 60.0, -10.0] (this lets the solver's output be independently sanity-checked, in addition to the formal verification in Section 5).

T_{desired}

0.26973	0.93649	-0.22412	-0.54158
---------	---------	----------	----------

-0.81068	0.09524	-0.57769	-0.30072
-0.51966	0.33751	0.78489	0.31700
0.00000	0.00000	0.00000	1.00000

IK result: converged = True, iterations = 13, solver final error norm = 3.586e-09

Solution joint vector q_{solution} (deg) = [15.0, 22.95, -70.4, 162.45, -60.0, 170.0]

Note the solution differs numerically from q_{true} — this is expected. The UR5 is kinematically redundant in representation (multiple joint configurations, e.g. elbow-up/elbow-down and wrist-flip solutions, can reach the same pose modulo 2π wraps), so IK correctness must be judged by pose reproduction, not by matching q_{true} exactly. This is exactly what Section 5 verifies.

5. IK Verification (Question 4)

The IK-derived joint angles are substituted back into forward kinematics to obtain T_{achieved} , which is then compared against T_{desired} .

T_{achieved} (FK of IK solution)

0.26973	0.93649	-0.22412	-0.54158
-0.81068	0.09524	-0.57769	-0.30072
-0.51966	0.33751	0.78489	0.31700
0.00000	0.00000	0.00000	1.00000

Metric	Value	Tolerance	Result
Position error	0.00000 mm	1.00 mm	PASS
Orientation error	0.00000 deg	0.50 deg	PASS

Overall IK solution validity: VALID — both position and orientation error are within the tolerances above, so the tool marks the solution as valid.

6. Jacobian Computation and Singularity Analysis

The 6x6 geometric Jacobian relates joint velocities to end-effector spatial velocity, $[v; \omega] = J(q) \cdot \dot{q}$. For an all-revolute manipulator, column i is:

$$J_{v,i} = z_{i-1} \times (p_e - p_{i-1}) \quad J_{\omega,i} = z_{i-1}$$

where z_{i-1} and p_{i-1} are the z-axis and origin of frame (i-1) expressed in the base frame (frame 0 has $z_0=[0,0,1]^T$, $p_0=0$).

Jacobian at Demonstration Configuration

q (deg) = [30.0, -45.0, 60.0, -90.0, 45.0, 20.0]

0.53787	-0.19980	0.06046	-0.02747	-0.04214	0.00000
-0.59693	-0.11536	0.03490	-0.01586	0.04287	0.00000

0.00000	-0.78589	-0.48537	-0.10649	0.05621	0.00000
0.00000	0.50000	0.50000	0.50000	-0.83652	0.19506
0.00000	-0.86603	-0.86603	-0.86603	-0.48296	-0.70388
1.00000	0.00000	0.00000	0.00000	-0.25882	0.68301

Cross-checked against a finite-difference (numerical) Jacobian of the position rows; maximum absolute discrepancy = 3.402e-07, confirming the analytical geometric Jacobian implementation is correct.

Singularity Identification

Singularities are identified from the singular values of J (SVD): a vanishing minimum singular value indicates loss of a degree of freedom. The manipulability index $w = \sqrt{\det(J J^T)}$ is reported alongside as a scalar closeness-to-singularity measure.

Configuration	Min. singular value	Manipulability	Singular?
Demonstration q	0.15726	0.078691	No
Straight arm, $q=[0,0,0,0,0,0]$	0.00000	0.000000	YES
Wrist-aligned, $q_5=0$	0.00000	0.000000	YES

As expected, the fully-extended (straight-arm) configuration and the wrist-aligned configuration ($\theta_5 = 0$, which lines up the joint-4 and joint-6 axes, the classic UR5 wrist singularity) both drive the smallest singular value to zero and are correctly flagged as singular, while the general demonstration configuration is not.

7. Conclusion

The implemented tool successfully models the UR5 using standard DH parameters, computes forward kinematics, solves inverse kinematics numerically via damped least-squares, verifies IK solutions by FK re-substitution with sub-millimetre / sub-degree accuracy, and correctly identifies kinematic singularities using the Jacobian singular values. All modules (dh.py, forward_kinematics.py, inverse_kinematics.py, jacobian.py, verification.py) are independent and reusable, and main.py demonstrates the complete pipeline end-to-end with numerical outputs saved to results/results.json and results/results.txt for validation.